Birzeit University

Department of Electrical & Computer Engineering

First Semester, 2025/2026

ENCS3130 Linux Laboratory

Python Project – Multi-Instructor Schedule Validation and Analysis System

---

1. Overview and Objectives

At Birzeit University, instructor teaching schedules are currently prepared manually and cover all instructional days except Friday and Sunday. Courses and laboratories are scheduled either on **Saturday, Monday, and Wednesday (S, M, W)** or on **Tuesday and Thursday (T, Th)**. Each instructor is required to carry a weekly teaching load ranging between **12 and 18 credit hours**, in accordance with university regulations. The objective of this project is to design and implement a **Python-based system** that automatically **parses, validates, and analyzes schedules for multiple instructors**. The system must ensure compliance with university policies, detect scheduling errors, and generate **instructor-specific statistical summaries and validation messages**. The project emphasizes **file processing, data validation, modular and object-oriented programming, and comprehensive reporting** using Python.

2. Input Files and Data Sources

The system operates on **multiple structured text files**, each serving a distinct role:

2.1 Instructor Schedule File

This file contains the schedules of **multiple instructors**. Each instructor's schedule begins with a **header line** identifying the instructor, followed by the weekly schedule lines.

Instructor Header Format

ID, FirstName, LastName

Weekly Schedule Format

Day | [Start–End]CourseOrLabCode; [Start–End]Activity; ...

- `Day` is one of the valid day abbreviations: `S`, `M`, `T`, `W`, `Th`
- Time slots are given in **hours or half-hours** (e.g., `8-9`, `9:30-11`)
- `CourseCode` represents a course identifier (e.g., `ENCS334`)
- `OH` represents Office Hours

For example, here is a sample schedule for instructor Ahmad:

120345, Ahmad, Saleh
S | [8–9]ENCS334; [9–11]OH; [12–1]ENCS533

M | [8–9]ENCS334; [12–1]ENCS233
T | [9:30–11]ENCS432; [11–12]OH
W | [8–9]ENCS334; [9–11]OH
Th| [9:30–11]ENCS432

2.2 Valid Course and Lab Codes File

This file contains the list of **all valid course and laboratory codes** offered by the department.

Format
ENCS211
ENCS313
...

The program must verify that every scheduled course or lab code exists in this file**.**

2.3 Instructor Preferences File

This file defines the **preferred courses and laboratories for each instructor**.

Format
InstructorID: CourseOrLabCode1; CourseOrLabCode2; ...

Example
120345: ENCS334; ENCS533; ENCS432
120412: ENCS211; ENCS311

The system must validate whether an instructor is assigned **only courses or labs listed in their preference file**, and report violations explicitly.

3. Required System Functionalities

Students must implement a **Python program** that processes all input files and performs the following tasks **independently for each instructor**, printing a clear, user-friendly message for every check.

- *Teaching Load Calculation*
  - Compute the **total teaching hours** (courses and labs only).
  - Compute the **total office hours (OH)**.
- *Teaching Load Validation:*
  - Verify that teaching load is within **12–18 hours**.
  - Output: *Valid Schedule, Invalid Schedule – Reason: …*
- *Office Hours Ratio Validation*
  - Ensure office hours ≥ **50% of teaching load**.
- *Teaching Days Distribution*
  - Confirm that teaching is distributed across **at least four distinct instructional days**.

- *Time Conflict Detection*
  - Detect **overlapping time slots** within the same day.
- *Consecutive Teaching Rule*
  - Ensure that **no more than two courses/labs** are scheduled consecutively without a break.
- *Allowed Teaching Days Validation*
  - Verify compliance with valid day groupings S, M, W or T, Th
- *Course and Lab Code Validation*
  - Ensure all scheduled courses/labs exist in the **valid codes file**.
- *Instructor Preference Validation*
  - Verify that assigned courses/labs match the instructor's **preference list**.

To fully exploit Python's capabilities, students must implement advanced features such as:

- ***Object-Oriented Architecture***
- ***Configurable Policy Engine: Store constraints (load limits, OH ratio, allowed days) in a configuration module or file.***
- ***Detailed Per-Instructor Reports: Generate a structured report per instructor including: Daily hour breakdown, Course repetition statistics, Validation summary***
- ***Robust Error Handling: Detect malformed input, missing instructors, invalid IDs, or inconsistent files without terminating execution.***

4. Submission Instructions

Submit a compressed .zip file containing:

- Python source code
- All required input files
- Sample test cases
- A README.md file explaining:
  - Program usage and execution
  - File formats
  - Implemented features and assumptions

5. Notes and Academic Policies

- The project must be implemented **entirely in Python**.
- Code must be clean, modular, well-documented, and properly indented.
- Students must work in **groups of no more than two**. *Individual submissions will result in a 30% grade deduction.*
- **Deadline:** Thursday, **15 January 2026**, at **11:59 PM**.
- Submission must be made via **Ritaj**.
- This is a **group project**; any academic dishonesty will result in failing the course.